Instructor: _____ Date: _____

OOPJ: Class Members, Objects, and References

INSTRUCTIONS: This faculty guide provides a complete lesson structure for teaching class members, objects, and references in Java to undergraduate CSE students. Use the opening question to engage students, follow the teaching sequence, and address common misconceptions through the practice questions provided.

1. Opening Question

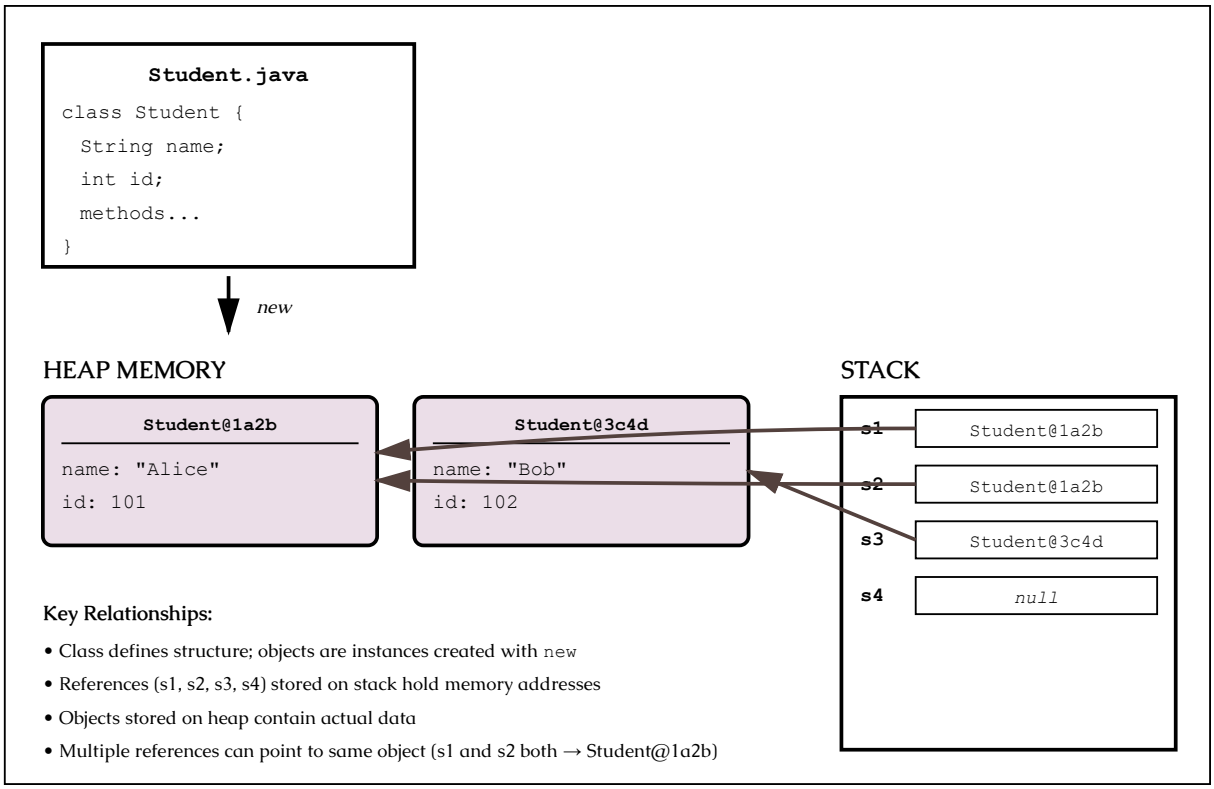
Ask students: "When you write `Student s1 = new Student();` and then `Student s2 = s1;`, how many Student objects exist in memory? What happens when you modify s2's name field?"

This exposes the critical distinction between reference variables and objects, setting up the lesson's core concept.

2. Teaching Sequence

1. **Class as a Blueprint:** Begin with the class definition as a template specifying instance variables (state) and methods (behavior). Emphasize that the class itself is not an object.
2. **Object Creation:** Explain that `new` allocates memory on the heap and invokes the constructor, returning a reference to that memory location.
3. **Reference Variables:** Clarify that variables of class type hold references (memory addresses), not the objects themselves. Java language rule: all object access is through references.
4. **Member Access:** Demonstrate dot notation (`obj.field`, `obj.method()`) to access instance members through a reference.
5. **Multiple References:** Show how multiple reference variables can point to the same object, and how this affects modifications.
6. **Null References:** Explain `null` as a reference pointing to no object, and the resulting `NullPointerException` when dereferencing.

3. Memory Model Diagram



4. Common Misconceptions

A. **Misconception:** "Assignment copies the object."

Expose with: "If `Student s1 = new Student("Alice");` and `Student s2 = s1;`, then `s2.setName("Bob");`, what is s1's name?"

Correct understanding: Assignment copies the reference, not the object. Both s1 and s2 point to the same object, so the name changes to "Bob" for both references. Java language rule: assignment of reference types copies the reference value.

B. **Misconception:** "Passing an object to a method creates a copy."

Expose with: "If you pass a Student object to a method that calls `setName()`, does the original object change?"

Correct understanding: Java is pass-by-value, but for objects, the value passed is the reference. The method receives a copy of the reference pointing to the same object, so modifications affect the original object. Simplified analogy: passing a house address (reference) vs. passing a copy of the house.

C. **Misconception:** "Comparing objects with == checks if their contents are equal."

Expose with: "If `Student s1 = new Student("Alice");` and `Student s2 = new Student("Alice");`, does `s1 == s2` return true?"

Correct understanding: The == operator compares reference values (memory addresses), not object contents. It returns true only if both references point to the exact same object. Use `.equals()` for content comparison (when properly overridden). Java language rule: == tests reference equality for objects.

5. Java Demonstration

Code Example:

```
class Student {
    String name;
    int id;

    Student(String name, int id) {
        this.name = name;
        this.id = id;
    }

    void display() {
        System.out.println(name + " (ID: " + id + ")");
    }
}

public class Demo {
    public static void main(String[] args) {
        Student s1 = new Student("Alice", 101);
        Student s2 = s1;
        Student s3 = new Student("Alice", 101);

        System.out.println("s1 == s2: " + (s1 == s2));
        System.out.println("s1 == s3: " + (s1 == s3));

        s2.name = "Bob";
        s1.display();
        s2.display();
        s3.display();
    }
}
```

Expected Output:

```
s1 == s2: true
s1 == s3: false
Bob (ID: 101)
Bob (ID: 101)
Alice (ID: 101)
```

⚠ *UNVERIFIED: This code has not been compiled or run. Verify before presenting to students.*

6. Practice Questions

1. **Basic:** What is printed by the following code?

```
Student s1 = new Student("John", 50);
Student s2 = new Student("John", 50);
System.out.println(s1 == s2);
```

Answer: `false`

Explanation: Two separate Student objects are created with `new`, so s1 and s2 hold different references (different memory addresses). The `==` operator compares references, not content, so even though both objects have identical field values, the comparison returns false.

2. **Intermediate:** Trace the following code and determine the final value of `s1.id`.

```
Student s1 = new Student("Alice", 10);
Student s2 = s1;
s2.id = 20;
Student s3 = new Student("Bob", 30);
s1 = s3;
s2.id = 40;
System.out.println(s1.id);
```

Answer: `30`

Explanation: Initially s1 and s2 reference the same object (Alice, 10). When `s2.id = 20`, that object becomes (Alice, 20). Then `s1 = s3` makes s1 point to the Bob object (Bob, 30), while s2 still points to the Alice object. When `s2.id = 40`, only the Alice object changes to (Alice, 40). The Bob object remains (Bob, 30), so `s1.id` is 30.

3. **Advanced:** Explain why the following method does not swap two Student objects as intended, and propose a fix.

```
void swap(Student a, Student b) {
    Student temp = a;
    a = b;
    b = temp;
}

// Called as:
Student s1 = new Student("X", 1);
Student s2 = new Student("Y", 2);
swap(s1, s2);
// s1 and s2 are unchanged
```

Answer: The method receives copies of the references s1 and s2. Swapping a and b only swaps the local copies inside the method; the original references s1 and s2 in the caller remain unchanged. Java is pass-by-value for all types, including references.

Fix: Java cannot swap object references through a method. Instead, swap the contents of the objects by exchanging their fields, or return a container holding both references, or use an array to hold the references and swap array elements.

7. Board Plan

Left Section: Class definition syntax with annotations

Center Section: Memory diagram showing stack (references) and heap (objects)

Right Section: Live code tracing area for student examples

8. Ten-Minute Teaching Rehearsal Outline

1. **0:00–1:00:** Pose opening question; take 2–3 student responses without confirming correctness.
2. **1:00–3:00:** Define class as blueprint; write simple Student class on board with two fields and constructor.
3. **3:00–5:00:** Create object with `new`; draw memory diagram showing reference variable on stack pointing to object on heap.
4. **5:00–7:00:** Add second reference to same object; demonstrate that modifying through one reference affects the other.
5. **7:00–8:30:** Introduce `null` and show attempted access causing NullPointerException.

6. **8:30–10:00:** Return to opening question; have students explain the answer using the memory model; address any remaining confusion.